

LC 778 - SWIM IN RISING WATER

Authored by: Arpit Raj, LNMIIT Jaipur | July 23, 2026

Companies: (DE Shaw, PhonePe, Uber)

Problem Notes & Concept

UNDERSTANDING THE GRID

- at time $t \Rightarrow$ water level is t
- grid represents elevation $\rightarrow$ if $val \leq t \rightarrow$ submerge in water
- we need to go from $(0, 0)$ to $(n-1, m-1) \rightarrow$ square
- in minimum time

CORE CONCEPT

t time me pohoch gaye to $(t+1)$, $(t+2)$... $(t+\infty)$ me bhi phoch jayenge hence suggesting binary search

SEARCH SPACE & CONSTRAINTS

- each value in a grid is unique
- $1 \leq n \leq 50 \rightarrow 2500$ cells at max
- hence we choose $l = 0$, $h = 2501 \rightarrow$ by 2501th minute the land will become a pool

TRAVERSAL LOGIC

Now while doing Binary Search
lets say $mid = t$

then while traversing we need to ensure that while going from (0,0) to (n-1, m-1) all cells are $\leq t$, otherwise it's not swimmable

C++ Solution Code

```
class Solution {
public:

    int n;
    int m;

    bool check(int mid, vector<vector<int>>& grid) {

        if(grid[0][0] > mid || grid[n-1][m-1] > mid){
            return false; //because these are guarenteed to be in path and agar
            yahi unswimmable ho gye to impossible
        }
        vector<vector<int>> seen(n, vector<int>(m, 0));
        vector<pair<int, int>> dir={{1,0}, {0,1}, {0,-1}, {-1,0}};

        queue<pair<int, int>> q;

        seen[0][0] = 1;
        q.push({0, 0});

        while(!q.empty()){
            int currsz = q.size();
            while(currsz--){

                int r = q.front().first;
                int c = q.front().second;
                if(r == n-1 && c == m-1) return true;
                q.pop();
                for(auto it : dir){
                    int nr = r + it.first;
                    int nc = c + it.second;

                    if(nr >= 0 && nc >= 0 && nr < n && nc < m){
                        if(grid[nr][nc] <= mid && !seen[nr][nc]){
                            seen[nr][nc] = 1;
                            q.push({nr, nc});
                        }
                    }
                }
            }
        }
    }
}
```

```

        return false;
    }

    int swimInWater(vector<vector<int>>& grid) {
        n = grid.size();
        m = grid[0].size();

        int ans = 0;
        int l = 0;
        int h = 2501;
        while(l <= h){
            int mid = l + (h-l)/2;

            if(check(mid, grid)){
                ans = mid;
                h = mid-1; //try karo aur kam baarish me bhi kaam done ho
            }
            else{
                l = mid+1; //itni baarish se kaam nhi chala
            }
        }

        return ans;
    }
};

```